

실습문제: 4장. 신경망 학습(2)

학번: 202404252 이름: 원종우

1. 다음 함수의 기울기(gradient)를 구하시오. 손 계산하시오.

$$f(x_0, x_1, x_2) = 1 + 2x_1 + x_0x_1x_2 + x_2^3$$

1번 문제. $f(x_0, x_1, x_2) = 1 + 2x_1 + x_0x_1x_2 + x_2^3$

i) $\frac{\partial f}{\partial x_0} = x_1x_2$ ii) $\frac{\partial f}{\partial x_1} = 2 + x_0x_2$ iii) $\frac{\partial f}{\partial x_2} = x_0x_1 + 3x_2^2$

$\therefore \nabla f(x_0, x_1, x_2) = (x_1x_2, 2 + x_0x_2, x_0x_1 + 3x_2^2)$

2. numerical_gradient 파이썬 함수를 이용해서, (0, 1, 2)에서 위 함수의 기울기를 구하시오. 코드와 출력결과를 캡처하여 넣으시오.

힌트: 다음 코드를 참고하시어 코드를 작성하고 실행시켜 출력 결과를 확인하세요.

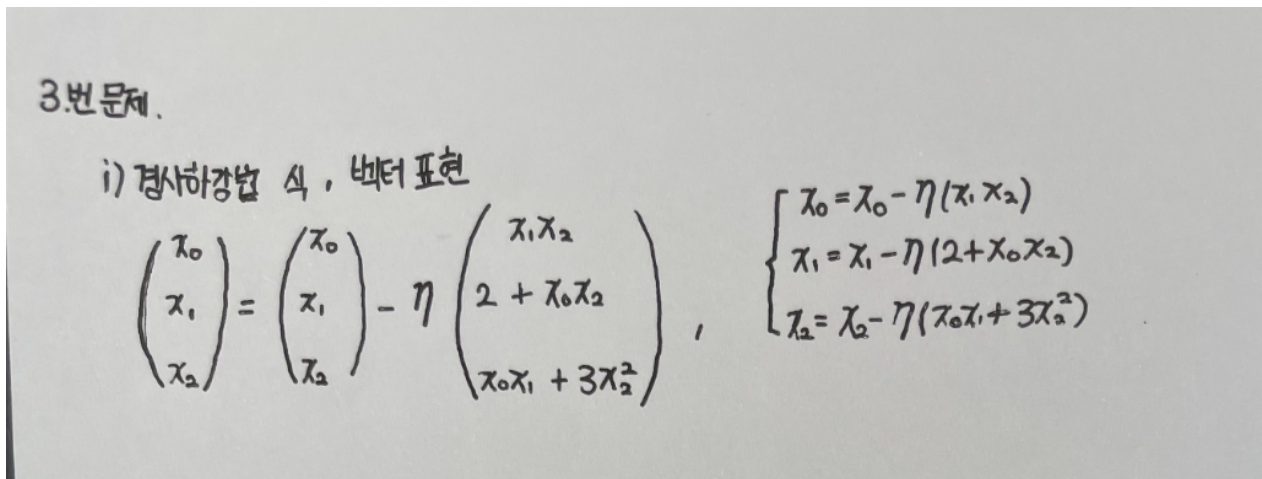
```
n2.py > ...
1  import numpy as np
2
3  def numerical_gradient(f, x):
4      h = 1e-4
5      grad = np.zeros_like(x) # x와 형상이 같은 배열을 생성
6
7      for idx in range(x.size):
8          tmp_val = x[idx]
9          # f(x+h) 계산
10         x[idx] = tmp_val + h
11         fxh1 = f(x)
12
13         # f(x-h) 계산
14         x[idx] = tmp_val - h
15         fxh2 = f(x)
16
17         grad[idx] = (fxh1 - fxh2) / (2 * h)
18         x[idx] = tmp_val # 값 복원
19
20     return grad
21
22 def function_f(x):
23     return 1+2*x[1]+x[0]*x[1]*x[2]+np.power(x[2],3.0)
24
25 x = np.array([0.0, 1.0, 2.0])
26 print(numerical_gradient(function_f, x))
```

```
> python3 ques_2.py
[ 2.          2.          12.00000001]

~/Doc/수 /2학 년 2/신 /실 습 자 료 /6주 차 | main ?1
```

3. 위 함수의 경사하강법 식을 적으시오. 벡터 표현으로도 적어보시오.

힌트:
$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix} - \eta \begin{pmatrix} x_1 x_2 \\ 2 + x_0 x_2 \\ x_0 x_1 + 3x_2^2 \end{pmatrix}$$



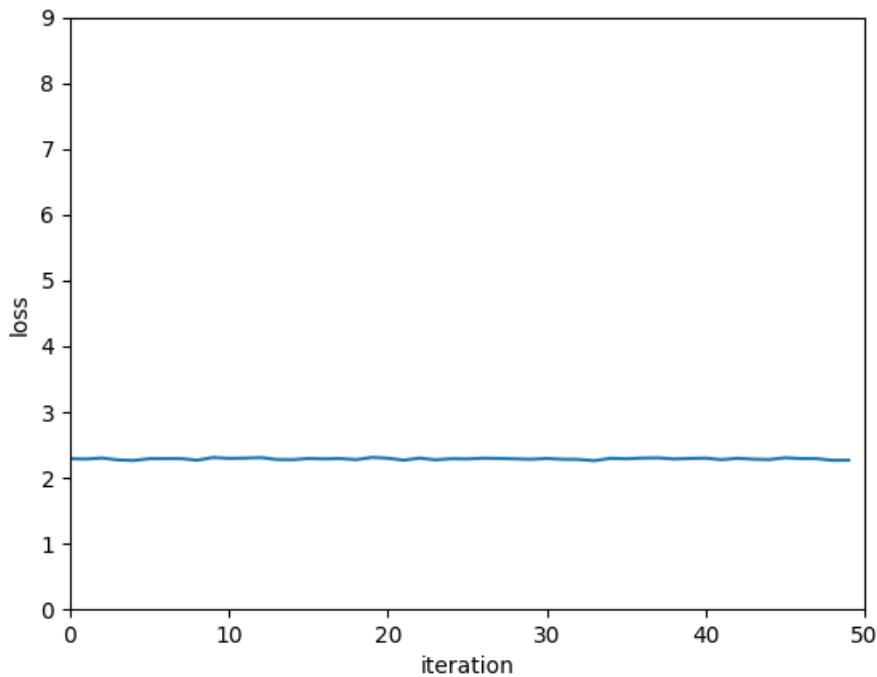
4. TwoLayerNet 클래스의 멤버 변수와 멤버 함수는 무엇인가?

멤버 변수: self.params (안에 W1, b1, W2, b2 포함)

멤버 함수: __init__, predict, loss, accuracy, numerical_gradient

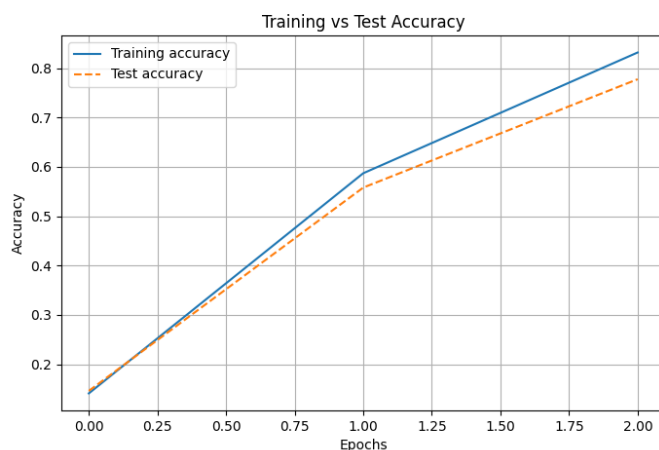
5. 교재 4.5.2절 코드에서, 학습 반복 횟수를 50으로 설정하고, 미니배치의 크기가 80일 때, 그림 4-11과 같은 손실함수 값의 추이를 볼 수 있는 그래프를 그리시오. 또, 다음 코드를 이용하여, 데이터 사이즈를 28×28가 아닌 7×7로 줄여 신경망에 입력하시오.

```
x_train = x_train[:, ::4]
x_test = x_test[:, ::4]
```



6. 60,000개의 훈련 데이터를 확률적 경사하강법으로 학습하려 한다. 미니배치의 크기가 200일 때 몇 번을 반복(iteration)하면 1 에폭(epoch)이 되는가? 이때, 6,000번 반복은 몇 에폭과 같은가?
 힌트: $200 \times x = 60,000$ 에서, $x = 300$ 이므로, 300 iteration을 돌리면(300번 반복하면) 1 epoch이다.
 300번

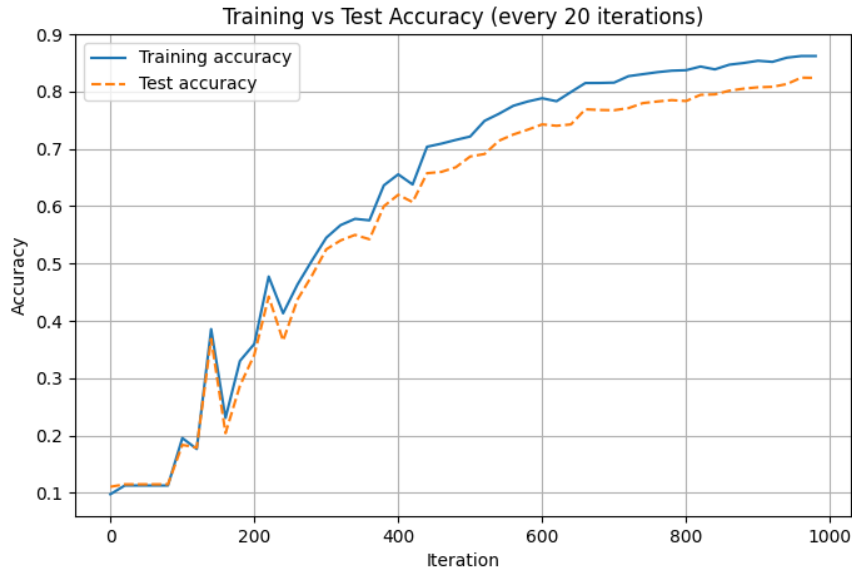
7. 교재 4.5.3절 코드를 실행하고 그림 4-12와 같은 그래프를 그리시오. 단, 반복 횟수는 1,000, 배치 사이즈는 80으로 설정하고, 데이터 사이즈는 위 5번을 참고하여 1/4로 사이즈를 줄이시오.



(추가문제)

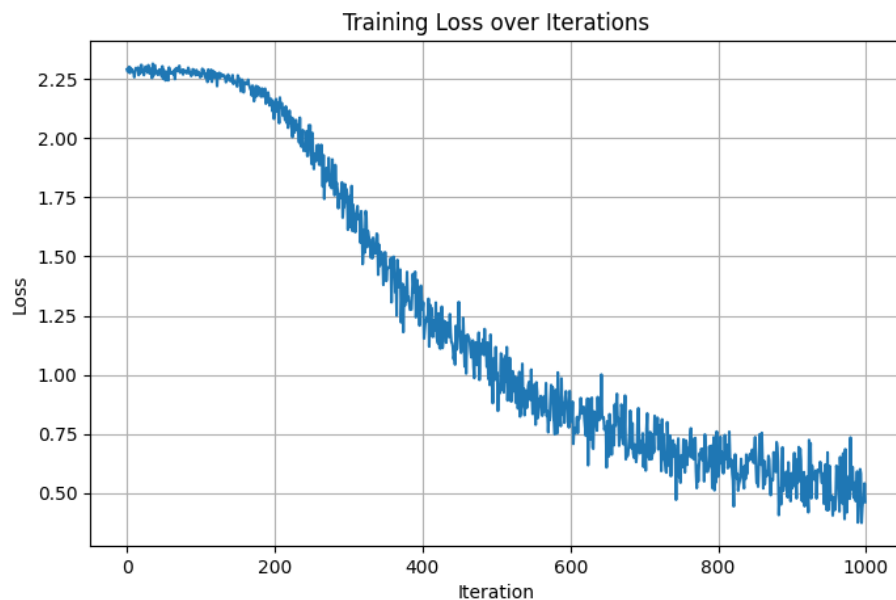
위 7번에 다음을 추가하시오.

- 매 20번 반복마다 accuracy를 저장하여, 그림 4-12와 같은 그래프를 그리시오.



20번 반복마다 정확도를 저장하고 시각화 해보니 모델의 정확도가 반복이 많을 수록 꾸준히 증가하는 것이 아닌 꺾이는 부분도 있다는것을 배웠습니다.

- 그림 4-11과 같은 loss 값 추이 그래프를 그리시오.



- 다음 코드를 참고하여 프로그램 수행 시간(속도)을 측정하시오. 몇 분 걸렸는가?
139.78717494010925초가 걸렸습니다.

힌트: 다음과 같이 time 모듈을 이용하면 수행시간을 계산할 수 있다.

```
import time
start_time = time.time()
수행 코드들 나열
```

```
end_time = time.time()  
print(end_time-start_time)
```